# Arshith Connect — Total Project Documentation

*Enterprise Human Resource Management System (HRMS) & Agile Project Operations Suite*

---

*Enterprise Human Resource Management System (HRMS) & Agile Project Operations Suite*

## 1. Executive Summary & Purpose

**Arshith Connect** is an integrated, enterprise-grade web application combining standard Human Resource Management (onboarding, payroll, attendance, leave management, compliance) with Agile Project Operations & Task Execution.

The platform links four primary stakeholder roles into a single, cohesive workflow:

1. **Administrator**: Global system oversight, audit logging, department management, payroll generation, and company settings.
2. **Project HR / Project Manager**: Project lifecycle creation with proposal PDF attachments, team formation, verification and assignment of approved Team Leaders, and project employee allocation.
3. **Team Leader**: Oversight of assigned projects, direct project-card task assignment, workload estimation, employee deliverable review, rework assignment with feedback, and task approvals.
4. **Employee**: Personal task boards, deadline tracking, work log submission with file/screenshot proof, review feedback inspection, and task resubmission.
5. **General HR**: Employee lifecycle operations, attendance tracking, leave requests, announcements, and separation procedures.

## 2. Technology Stack Breakdown

Layer • Technology • Key Libraries & Purpose

**Frontend Framework** • **React 18** (SPA) • Modern component architecture, functional hooks (`useState`, `useEffect`, `useMemo`, `useCallback`).

**Build Tool & Bundler** • **Vite 5 / 8** • High-performance ES module bundler, hot module replacement (HMR), static asset compilation (`dist/`).

**Routing** • **React Router DOM v6** • Client-side routing, protected routes (`AdminRoute`, `HrRoute`, `TeamLeaderRoute`, `EmployeeRoute`).

**Styling & UI Design** • **Tailwind CSS + Custom CSS** • Tailored corporate emerald branding (`#0b3d2b`, `#0f766e`), responsive cards, CSS flex/grid layouts.

**Iconography** • **Lucide React** • Feather-derived icons (`FolderKanban`, `Users`, `CheckCircle2`, `FileText`, `Plus`, `RefreshCw`, etc.).

**Backend Framework** • **Python 3.12 + FastAPI** • High-throughput asynchronous REST API framework with automatic OpenAPI/Swagger documentation.

**ASGI Web Server** • **Uvicorn** • Asynchronous Server Gateway Interface for running FastAPI applications in production and development.

**Data Validation** • **Pydantic v2** • Strict schema validation, request/response models, type coercion, and error messages.

**Authentication & AuthZ** • **PyJWT + Passlib (bcrypt)** • Stateless JWT tokens (HS256 algorithm), salted password hashing, Bearer authentication scheme.

**Database Engines** • **Dual Engine Architecture** • **Google Cloud Firestore**: Primary document database via `firebase-admin` SDK.<br>**Local JSON Store (`firestore_local_db.json`)**: Automatic zero-cloud fallback for offline or air-gapped development.<br>**PostgreSQL + SQLAlchemy**: Optional relational database mapping.

**Static Hosting** • **FastAPI StaticFiles Mount** • Single-port fullstack architecture: FastAPI serves the compiled React application directly from `frontend/dist` at `/`.

## 3. High-Level System Architecture

```
flowchart TD
    subgraph ClientLayer["Frontend Client (React 18 SPA)"]
        UI_Admin["Admin Portal"]
        UI_ProjHR["Project HR Portal"]
        UI_TL["Team Leader Portal"]
        UI_Emp["Employee Portal"]
        UI_Auth["Auth & Registration"]
    end

    subgraph APILayer["FastAPI Gateway (Port 8000)"]
        Router_Auth["/api/auth/*"]
        Router_Projects["/api/projects/*"]
        Router_Tasks["/api/employee-tasks/*"]
        Router_TL["/api/team-leader/*"]
        Router_HR["/api/employees/* & /api/hr/*"]
        Router_Static["/ (StaticFiles -> frontend/dist)"]
    end

    subgraph ServiceLayer["Core Backend Business Services"]
        Auth_Service["Security & JWT Engine"]
        Project_Service["Project & Allocation Engine"]
        Task_Service["Task & Deliverable Engine"]
        Notification_Service["Notification Hub"]
        Firestore_Service["Firestore Local/Cloud Adapter"]
    end

    subgraph DataLayer["Persistence Stores"]
        Cloud_Firestore[("Google Cloud Firestore")]
        Local_JSON[("Local JSON DB\n(firestore_local_db.json)")]
        PostgreSQL_DB[("PostgreSQL Database\n(SQLAlchemy)")]
    end

    ClientLayer -->|REST HTTPS Requests + JWT| APILayer
    APILayer --> ServiceLayer
    ServiceLayer --> Firestore_Service
    Firestore_Service -->|Cloud Credentials Available| Cloud_Firestore
    Firestore_Service -->|Local Store Fallback| Local_JSON
    ServiceLayer -.->|Optional Relational Mode| PostgreSQL_DB
```

## 4. Directory & Codebase Structure Explanation

```
ArshithConnect-Web-main/
■■■ backend/                        # Python FastAPI Backend Service
■   ■■■ app/
■   ■   ■■■ core/                    # Core Security, Config & Firebase Setup
■   ■   ■   ■■■ config.py            # Environment variables, JWT secret, database configs
■   ■   ■   ■■■ exceptions.py        # Custom HTTP error handlers
■   ■   ■   ■■■ firebase.py          # Firebase Admin SDK initialization
■   ■   ■   ■■■ security.py          # Bcrypt password hashing & JWT encoding/decoding
■   ■   ■■■ db/                      # Relational DB setup (SQLAlchemy)
■   ■   ■   ■■■ base.py              # Declarative base model
■   ■   ■   ■■■ session.py           # SessionLocal database session generator
■   ■   ■■■ dependencies/           # FastAPI Route Injection Dependencies
■   ■   ■   ■■■ auth.py              # get_current_user, require_role
■   ■   ■■■ models/                  # SQLAlchemy ORM Data Models (for PostgreSQL mode)
■   ■   ■   ■■■ announcement.py, assignment.py, attendance.py, employee.py, ...
■   ■   ■   ■■■ user.py              # User authentication model
■   ■   ■■■ schemas/                 # Pydantic Schemas (Request/Response contracts)
■   ■   ■   ■■■ auth.py, employee.py, leave.py, meeting.py, team.py, work_update.py, ...
■   ■   ■■■ services/                # Core Domain Business Logic
■   ■   ■   ■■■ firestore_service.py # Unified Firestore & Local JSON Store Abstraction
■   ■   ■   ■■■ init_firestore.py    # Database seeder & default schema initialization
■   ■   ■   ■■■ payroll_engine.py    # Gross-to-net salary computation engine
■   ■   ■   ■■■ payslip_pdf.py       # PDF payslip generator
■   ■   ■   ■■■ sync_service.py      # Bi-directional Firestore/PostgreSQL synchronization
■   ■   ■■■ routers/                 # 31 API Controller Endpoints
■   ■   ■   ■■■ auth.py              # Login, registration, password management
■   ■   ■   ■■■ projects.py          # Project CRUD, approved TL/Employee endpoints, proposal PDFs
■   ■   ■   ■■■ employee_tasks.py    # Task creation, submission, review, rework, approvals
■   ■   ■   ■■■ team_leader.py       # Team Leader roster, assigned staff & performance stats
■   ■   ■   ■■■ employees.py         # Employee CRUD, profiles, approval status
■   ■   ■   ■■■ payroll.py           # Salary structures, bank export, payout runs
```

```
■   ■   ■   ■■■ attendance.py       # Check-in/out, geofencing, facial capture
■   ■   ■   ■■■ ...                 # announcements, chat, leaves, loans, meetings, etc.
■   ■   ■■■ main.py                 # FastAPI Application Entrypoint & Static Frontend Mounter
■   ■■■ firestore_local_db.json     # Primary local JSON document database
■   ■■■ requirements.txt            # Python package dependencies
■   ■■■ test_e2e_integration.py     # Comprehensive automated end-to-end test suite
■
■■■ frontend/                       # React 18 Single Page Application (SPA)
■   ■■■ src/
■   ■   ■■■ assets/                 # Brand assets, corporate logos, vector graphics
■   ■   ■■■ components/             # Reusable UI Components
■   ■   ■   ■■■ common/             # Button, Modal, DataTable, Badge, Card, StatCard
■   ■   ■   ■■■ layout/             # TopNavbar, Sidebar, Footer, PageHeader
■   ■   ■■■ layouts/                # Portal-specific layouts
■   ■   ■   ■■■ AdminLayout.jsx     # Administrator container layout
■   ■   ■   ■■■ EmployeeLayout.jsx  # Employee portal layout
■   ■   ■   ■■■ HrLayout.jsx        # General HR portal layout
■   ■   ■   ■■■ HrProjectsLayout.jsx# Project HR portal layout
■   ■   ■   ■■■ TeamLeaderLayout.jsx# Team Leader portal layout
■   ■   ■■■ pages/                  # Portal Page Views
■   ■   ■   ■■■ admin/              # AdminDashboard, AdminProjects, AdminEmployees, AdminPayroll...
■   ■   ■   ■■■ hr-projects/        # HrProjectsDashboard (Project creation, proposal PDF, TL assign)
■   ■   ■   ■■■ team-leader/        # TeamLeaderDashboard (Project roster, +Add Task, Staff tray)
■   ■   ■   ■■■ employee/           # EmployeeWorkAssignments, EmployeeDashboard, EmployeeTasks...
■   ■   ■   ■■■ hr/                 # HrDashboard, HrEmployees, HrAttendance, HrLeaves...
■   ■   ■   ■■■ auth/               # Login, HrProjectsLogin, Register, TeamLeaderRegister
■   ■   ■■■ routes/                 # Protected Route Guards & AppRoutes.jsx
■   ■   ■■■ services/               # 30 Axios API Client Wrappers
■   ■   ■   ■■■ api.js              # Base Axios instance with Bearer token interceptor
■   ■   ■   ■■■ projectService.js   # Project CRUD, TL assignment, approved staff retrieval
■   ■   ■   ■■■ employeeTaskService.js # Task assignments, deliverable upload, rework cycles
■   ■   ■   ■■■ authService.js      # Session management & token decoding
■   ■   ■   ■■■ ...                 # attendance, payroll, leaves, announcements, etc.
■   ■   ■■■ utils/                  # Formatters, date helpers, role permission constants
■   ■■■ package.json                # Frontend package dependencies & scripts
■   ■■■ vite.config.js              # Vite bundling & proxy configurations
■   ■■■ tailwind.config.js          # Tailwind CSS styling tokens & themes
■
■■■ run-project.bat                 # One-click Windows startup script
■■■ README.md                       # Repository instructions
```

## 5. Core Portals & Module Breakdown

### 5.1 Administrator Portal (`/admin/`)

• **System Control & Access Control**: Complete visibility over all registered users across all roles (`admin`, `hr`, `hr_projects`, `team_leader`, `employee`).

• **Project Operations Oversight**: Directly mounts `AdminProjects.jsx`, allowing administrators full Project HR capabilities (creating projects, attaching proposal PDFs, approving team leaders, monitoring progress).

• **Comprehensive HRMS Controls**: Attendance logs, leaves, payroll processing, separation workflows, recruitment pipeline, announcements, and meeting schedules.

### 5.2 Project HR / Operations Portal (`/hr-projects/`)

• **Project Creation & Proposal Upload**:

• Replaced legacy department selectors with an **Upload Project Proposal PDF** control (`.pdf`).

• Stores documents safely as Base64 data URLs in local mode or Cloud Storage URLs in cloud mode.

• **Approved Team Leader Allocation**:

• The **"Assign Team Leader"** dropdown strictly queries `GET /api/projects/approved-leaders`.

• Filters exclusively for verified team leaders (`approval_status == 'APPROVED'` and `is_active == True`).

• **Approved Employee Team Formation**:

• The employee selector allows multi-selection of registered, approved active staff, preventing dummy or administrative users from being assigned to project rosters.

• **Proposal PDF Viewer**:

• Every project card in "All Projects" displays a dedicated **"View Proposal PDF"** button that opens the proposal in an isolated, styled PDF reader tab.

### 5.3 Team Leader Portal (`/teamlead/`)

• **Assigned Projects Roster**:

• Team leaders see only the projects delegated specifically to them by Project HR.

• **Project-Card Based Task Creation**:

• Each project card features a dedicated `+ Add Task` button.

• Clicking this button locks the task directly to that project (eliminating any redundant project dropdown selection).

• The employee dropdown is scoped strictly to the team members assigned to that project.

• **Collapsible Staff Roster Tray**:

• `View Assigned Employees` button opens a staff tray with an individual `Assign Task` action button for each team member.

• **Deliverable Review & Approval Pipeline**:

• Team leaders review submitted employee task deliverables and screenshots.

• Actions: **Approve Task** (100% completion) or **Assign Rework** (reverts status to `REWORK` with compulsory feedback notes).

### 5.4 Employee Portal (`/employee/`)

• **Active Project Display & Proposal PDF Access**:

• Displays the active project name, description, assigned team leader, and an inline **"View Project Proposal PDF"** button.

• **Personal Task Board**:

• Employees see only tasks assigned directly to them under their project.

• Displays task status (`NOT_STARTED`, `IN_PROGRESS`, `SUBMITTED`, `REWORK`, `APPROVED`).

• **Work Submission & Daily Work Logs**:

• Form to submit task deliverables with work descriptions and proof attachments/screenshots.

• Daily work update log recording hours worked and blockers.

• **Rework & Feedback Loop**:

• If a task is rejected/rework requested, the leader's specific feedback is highlighted in an amber notice banner with a form to resubmit revised work.

---

## 6. Key Workflows & Lifecycles

---

### 6.1 Team Leader Registration & Admin Approval Flow

```
sequenceDiagram
    autonumber
    actor TL as Candidate Team Leader
    actor HR as Project HR / Admin
    participant Auth as Auth Router (/api/auth)
    participant Proj as Project Router (/api/projects)
    participant DB as Firestore DB

    TL->>Auth: POST /api/auth/register-team-leader (Registration form)
    Auth->>DB: Save to 'team_leaders' (approval_status: 'PENDING', is_active: false)
    Auth-->>TL: 201 Created (Pending Admin Approval)
    Note over TL,Auth: Candidate cannot log in or appear in project assignment dropdowns

    HR->>Proj: GET /api/projects/team-leaders/approvals
    Proj->>DB: Query 'team_leaders' collection
    DB-->>Proj: Return pending candidates
    Proj-->>HR: Display in "Team Leader Approvals" table

    HR->>Proj: PATCH /api/projects/{id}/team-leader-approval (status: 'APPROVED')
    Proj->>DB: Update 'team_leaders' & 'users' (approval_status: 'APPROVED', is_active: true)
    Proj-->>HR: 200 OK (Status Approved)

    HR->>Proj: GET /api/projects/approved-leaders
    Proj->>DB: Query team_leaders WHERE status == 'APPROVED' AND is_active == true
    DB-->>Proj: [keerthana, teamlead, ...]
    Proj-->>HR: Render in "Assign Team Leader" Dropdown
```

### 6.2 Project Creation, PDF Proposal & Task Execution Workflow

```
sequenceDiagram
    autonumber
    actor HR as Project HR
    actor TL as Team Leader
    actor Emp as Employee
    participant API as FastAPI Backend
    participant DB as Firestore DB

    HR->>API: POST /api/projects (Name, Proposal PDF, Approved TL, Approved Staff)
    API->>DB: Store project in 'projects' collection
    API-->>HR: 201 Created (Project Assigned)

    TL->>API: GET /api/team-leader/projects
    API->>DB: Query projects WHERE team_leader == TL.name
    API-->>TL: Return assigned projects with Proposal PDF

    Note over TL: Clicks "+ Add Task" directly on the Project Card
    TL->>API: POST /api/team-leader/tasks (project_id, employee, deadline, title)
    API->>DB: Store in 'project_tasks' (status: 'NOT_STARTED')
    API-->>TL: 201 Created (Task Assigned)

    Emp->>API: GET /api/employee-tasks/my-tasks
    API->>DB: Query tasks WHERE assigned_to == Emp.name
    API-->>Emp: Display task on board

    Emp->>API: POST /api/employee-tasks/{id}/submit (Work notes + screenshot proof)
    API->>DB: Update task (status: 'SUBMITTED') & Send notification to TL
    API-->>Emp: 200 OK (Submission Received)

    alt Leader Approves Task
        TL->>API: PATCH /api/employee-tasks/{id}/review (action: 'APPROVE')
        API->>DB: Update task (status: 'APPROVED', progress: 100)
        API-->>TL: 200 OK (Task Completed)
    else Leader Requests Rework
        TL->>API: PATCH /api/employee-tasks/{id}/review (action: 'REWORK', feedback: '...')
        API->>DB: Update task (status: 'REWORK') & Send feedback notification to Emp
        API-->>TL: 200 OK (Rework Assigned)
        Emp->>API: POST /api/employee-tasks/{id}/resubmit (Revised proof)
        API->>DB: Update task (status: 'SUBMITTED')
    end
```

## 7. Database Collections & Schema Definitions

The platform operates on a document-oriented model stored in `backend/firestore_local_db.json` (or live Cloud Firestore):

### 1. `projects`

```
{
  "id": "PRJ-103647",
  "name": "AI Facial Recognition System Project",
  "description": "Sprint deliverable execution and milestones.",
  "deadline": "2026-11-20",
  "priority": "High",
  "status": "In Progress",
  "progress": 0,
  "team_leader": "keerthana",
  "team_leader_id": "AFI-9060",
  "assigned_employees": ["hema", "keerthana1"],
  "proposal_pdf": "data:application/pdf;base64,JVBERi0xLjQK...",
  "proposal_pdf_name": "AI_Facial_Recognition_Proposal.pdf",
  "created_at": "2026-11-04T10:45:00Z"
}
```

### 2. `team_leaders`

```
{
  "id": "AFI-9060",
  "team_leader_id": "AFI-9060",
  "full_name": "keerthana",
  "email": "keerthana@arshithgroup.com",
  "department": "Operations",
  "specialization": "Technical Leadership",
  "approval_status": "APPROVED",
  "is_active": true,
  "created_at": "2026-11-03T10:00:00Z"
}
```

### 3. `project_tasks`

```
{
  "id": "TSK-724596",
  "project_id": "PRJ-103647",
  "project_name": "AI Facial Recognition System Project",
  "title": "Cloud Tenant Isolation Testing",
  "description": "Execute security isolation test suites across AWS and Azure tenants.",
  "assigned_to": "hema",
  "assigned_to_id": "EMP-4512",
  "team_leader": "keerthana",
  "team_leader_id": "AFI-9060",
  "deadline": "2026-11-15",
  "priority": "High",
  "status": "APPROVED",
  "progress": 100,
  "submission_notes": "All tenant tests passed with zero leakage.",
  "rework_feedback": null,
  "attachments": [{"name": "test_matrix.png", "url": "..."}],
  "created_at": "2026-11-04T11:00:00Z"
}
```

### 4. `users`

```
{
  "id": "USR-9060",
  "email": "keerthana@arshithgroup.com",
  "role": "team_leader",
  "approval_status": "APPROVED",
  "is_active": true,
  "hashed_password": "$2b$12$..."
}
```

### 5. `employee_notifications`

```
{
  "id": "NOTIF-482910",
  "recipient_id": "AFI-9060",
  "type": "TASK_SUBMISSION",
  "title": "Task Submitted: Cloud Tenant Isolation Testing",
  "message": "hema has submitted deliverables for review.",
  "task_id": "TSK-724596",
  "is_read": false,
  "created_at": "2026-11-04T11:30:00Z"
}
```

## 8. REST API Endpoints Reference

### Authentication (`/api/auth`)

- `POST /api/auth/login`: Standard employee authentication.
- `POST /api/auth/admin-login`: Administrator credentials check.
- `POST /api/auth/hr-login`: General HR authentication.
- `POST /api/auth/hr-projects-login`: Project HR / Manager authentication.
- `POST /api/auth/register`: Employee self-registration (`PENDING` status).
- `POST /api/auth/teamlead/register`: Team leader candidate registration (`PENDING` status).

### Project Operations (`/api/projects`)

- `GET /api/projects`: List projects with status and department filters.
- `POST /api/projects`: Create project with proposal PDF and approved staff.
- `GET /api/projects/{id}`: Detailed project information.
- `PUT /api/projects/{id}`: Update project details, deadlines, and proposal documents.
- `DELETE /api/projects/{id}`: Remove project.
- `GET /api/projects/approved-leaders`: Return only registered, admin-approved team leaders.
- `GET /api/projects/approved-employees`: Return only approved, active staff members.
- `PATCH /api/projects/{id}/assign-leader`: Assign an approved team leader to a project.
- `PATCH /api/projects/{id}/assign-employees`: Assign approved staff members to a project.
- `GET /api/projects/team-leaders/approvals`: List pending and historical team leader approvals.
- `PATCH /api/projects/{id}/team-leader-approval`: Approve or reject a team leader registration.

### Team Leader Execution (`/api/team-leader`)

- `GET /api/team-leader/projects`: Retrieve projects delegated to the current team leader.
- `GET /api/team-leader/my-team`: Retrieve staff members assigned to the leader's projects.
- `GET /api/team-leader/tasks`: Retrieve all tasks created under the leader's projects.
- `POST /api/team-leader/tasks`: Create and assign a task directly to a project member.
- `GET /api/team-leader/performance-roster`: Completion rates and team task metrics.

### Employee Task Management (`/api/employee-tasks`)

- `GET /api/employee-tasks/my-tasks`: Retrieve tasks assigned to the logged-in employee.
- `GET /api/employee-tasks/my-active-project`: Retrieve current project details and proposal PDF.
- `PATCH /api/employee-tasks/{id}/status`: Update task progress percentage and status.
- `POST /api/employee-tasks/{id}/submit`: Submit task deliverables with proof attachments.
- `PATCH /api/employee-tasks/{id}/review`: Leader review (approve or assign rework with notes).
- `POST /api/employee-tasks/{id}/resubmit`: Employee resubmission following rework.
- `POST /api/employee-tasks/daily-work-update`: Submit daily log of hours and deliverables.
- `GET /api/employee-tasks/notifications`: In-app updates for task feedback and status changes.

## 9. Security, Validation & Error Handling

1. **Strict Input Sanitization & Type Coercion**:

• Pydantic models validate all incoming request bodies with custom field validators.

• Non-empty string constraints prevent empty task descriptions or invalid deadlines.

2. **Access Control & Multi-Tenant Scoping**:

• `get_current_user` extracts and cryptographically verifies the JWT token from the `Authorization: Bearer <token>` header.

• Team Leaders can only assign tasks to employees assigned to their specific projects.

• Employees cannot access or modify tasks belonging to other staff members.

3. **Data Integrity & Seeder Protection**:

• `init_firestore.py` ensures that dummy/sample users (`TL-101`, `TL-102`, sample projects) are never re-seeded into the operational database.

• All dashboard statistics are calculated dynamically via database queries rather than hardcoded counter values.

## 10. Setup, Configuration & Execution Guide

### Prerequisites

• **Python**: Version 3.10+ (Python 3.12 recommended)

• **Node.js**: Version 18+ or 20+

• **NPM**: Version 9+

### Quick Start (Windows)

Double-click `run-project.bat` in the project root, or execute via PowerShell:

```
# 1. Build the React frontend production bundle
cd frontend
npm install
npm run build
cd ..

# 2. Start the unified FastAPI web server
cd backend
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

### Accessing the Portals

Once the server is running on `http://localhost:8000`:

• **Administrator Portal**: `http://localhost:8000/admin` (Credentials: `admin@arshith.com` / `Password@123`)

• **Project HR Portal**: `http://localhost:8000/hr-projects/dashboard` (Credentials: `project.hr@arshith.com` / `HrProject@123`)

• **Team Leader Portal**: `http://localhost:8000/teamlead/dashboard` (Credentials: Registered approved TL / `TeamLead@123`)

• **Employee Portal**: `http://localhost:8000/employee/dashboard` (Credentials: Approved employee / `hema@123`)

• **Interactive Swagger API Docs**: `http://localhost:8000/docs`

*Documentation compiled for Arshith Connect. System operational on unified port 8000.*